

Workspace Assignment -

- You are managing infrastructure for multiple environments (dev, staging, and prod) using Terraform workspaces.
- Your goal is to create an S3 bucket and an EC2 instance, where:
 - The S3 bucket name should be prefixed with the workspace name.
 - The EC2 instance type should vary based on the workspace (t2.micro for dev, t3.small for staging, and t3.medium for prod).
- Store Terraform state in an S3 backend with state locking enabled using a DynamoDB table.
- Use variables for flexibility, locals for calculated values, and functions to dynamically construct names.

Creating workspaces:

terraform workspace new prod

terraform workspace new stage

terraform workspace new dev

To Select:

Terraform workspace select <workspace name>

Terraform workspace select prod

To List:

Terraform workspace list

Code:

The screenshot shows the VS Code interface with a Terraform project. The Explorer on the left shows a workspace with folders for Python, Terraform, and Terraform Ex2. Under Terraform Ex2, there is a terraform folder containing terraform.tfstate.d, which has subfolders dev, prod, and stage. Each subfolder contains terraform.tfstate, terraform.tfvars, and variables.tf files. The main.tf file is selected and open in the editor. The code in main.tf defines an AWS provider, a local workspace, common tags, and resources for an AWS instance and an S3 bucket. The terminal at the bottom shows the command 'terraform init' and its output.

```
1 provider "aws" {
2   #Access key and secret key are not needed
3   #refers from the environment variables
4   region = "us-east-2" // var.aws_region
5 }
6
7 locals {
8   workspace = terraform.workspace
9   bucket_name = "${local.workspace}-tf-bucketzhast21"
10  instance_types = var.instance_type
11
12  common_tags = {
13    Environment = local.workspace
14    ManagedBy = "Terraform"
15  }
16 }
17
18 resource "aws_instance" "Assignment" {
19   instance_type = lookup(var.instance_type, terraform.workspace, "t2.micro")
20   ami = var.ami_value
21
22   tags = merge(local.common_tags, {
23     Name = "${local.workspace}-EC2"
24   })
25 }
26
27 resource "aws_s3_bucket" "s3-bucket" {
28   bucket = local.bucket_name
29   tags = merge(local.common_tags, {
30     Name = local.bucket_name
31   })
32 }
33
34 # Create a DynamoDB table for state locking
35 resource "aws_dynamodb_table" "terraform_lock" {
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
+ versioning (known after apply)
+ website (known after apply)
}
```

Plan: 3 to add, 0 to change, 0 to destroy

The screenshot shows the VS Code interface with the backend.tf file open in the editor. The code defines the Terraform backend as S3, with a bucket name, key, region, and encrypt flag. It also defines a dynamodb_table for state locking.

```
1 terraform {
2   backend "s3" {
3     bucket = "my-tf-test-bucketzzz" # change this
4     key = "state/${terraform.workspace}/terraform.tfstate"
5     region = "us-east-2"
6     encrypt = true
7     dynamodb_table = "terraform-lock"
8   }
9 }
10 }
```

```

> Untitled-1  main.tf  backend.tf  variables.tf  terraform.tfvars
Terraform > Terraform Ex2 > variables.tf > variable "instance_type" > default
1  variable "ami_value"{
2      description = "ami value"
3      type = string
4  }
5
6  variable "instance_type" {
7      description = "instace type"
8      type = map(string)
9
10     default = {
11         "default" = "t2.micro"
12         "dev"     = "t2.medium"
13         "stage"   = "t2.micro"
14         "prod"    = "t2.large"
15     }
16

```

```

> Untitled-1  main.tf  backend.tf  variables.tf  terraform.tfvars
Terraform > Terraform Ex2 > terraform.tfvars > ...
1  ami_value = "ami-0cb91c7de36eed2cb"
2  //instance_type = "t2.micro"

```

Inside Workspace DEV

```
PS C:\devops\Terraform\Terraform Ex2> terraform workspace list
default
* dev
prod
stage

PS C:\devops\Terraform\Terraform Ex2> terraform workspace select dev
PS C:\devops\Terraform\Terraform Ex2> terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v5.89.0
```

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

```
PS C:\devops\Terraform\Terraform Ex2>
```

```
PS C:\devops\Terraform\Terraform Ex2> terraform plan
```

Terraform used the selected providers to generate the following execution plan. Resources to be added, changed, or destroyed are indicated by the following symbols:

- + create

Terraform will perform the following actions:

- # aws_dynamodb_table.terraform_lock will be created

Plan: 3 to add, 0 to change, 0 to destroy.

Changes to Outputs:

- + ec2_instance_type = "t2.medium"
- + s3_bucket_name = (known after apply)

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee it's accurate. To run "terraform apply" now.

```
PS C:\devops\Terraform\Terraform Ex2> terraform apply
```

Terraform used the selected providers to generate the following execution plan. Resources to be added, changed, or destroyed are indicated by the following symbols:

- + create

Outputs:

```
ec2_instance_type = "t2.medium"
s3_bucket_name = "dev-tf-bucketzhzast21"
```

Output:

☐	dev-EC2	i-029c873272a8c168d	Running	t2.medium	Initializing
--------------------------	---------	---------------------	---------	-----------	--------------

Inside Workspace STAGING:

```
PS C:\devops\Terraform\Terraform Ex2> terraform workspace list
default
* dev
prod
stage

PS C:\devops\Terraform\Terraform Ex2> terraform workspace select stage
Switched to workspace "stage".
PS C:\devops\Terraform\Terraform Ex2> terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v5.89.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\devops\Terraform\Terraform Ex2> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated
by the following symbols:
+ create
```

Outputs:

```
ec2_instance_type = "t2.micro"
s3_bucket_name = "stage-tf-bucketzhzast21"
PS C:\devops\Terraform\Terraform Ex2>
```

Output:

☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone
☐	stage-EC2	i-084059a1f5bf1dc84	Running	t2.micro	2/2 checks passed	View alarms +	us-east-2a
☐	dev-EC2	i-029c873272a8c168d	Running	t2.medium	2/2 checks passed	View alarms +	us-east-2a

Inside Workspace PROD:

```
PS C:\devops\Terraform\Terraform Ex2> terraform workspace select prod
Switched to workspace "prod".
PS C:\devops\Terraform\Terraform Ex2> terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v5.89.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\devops\Terraform\Terraform Ex2> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions
following symbols:
+ create

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:

ec2_instance_type = "t2.large"
s3_bucket_name = "prod-tf-bucketzhzast21"
PS C:\devops\Terraform\Terraform Ex2>
```

Output:

☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability
☐	prod-EC2	i-031837bc4f85131e5	Running	t2.large	Initializing	View alarms +	us-east-2a
☐	stage-EC2	i-084059a1f5bf1dc84	Running	t2.micro	2/2 checks passed	View alarms +	us-east-2a
☐	dev-EC2	i-029c873272a8c168d	Running	t2.medium	2/2 checks passed	View alarms +	us-east-2a

General purpose buckets

Storage buckets

General purpose buckets (3) Info All AWS Regions

Buckets are containers for data stored in S3.

Find buckets by name

	Name	▲	AWS Region	▼	IAM Access Analyzer
☐	dev-tf-bucketzhzast21		US East (Ohio) us-east-2		View analyzer for us-east-2
☐	prod-tf-bucketzhzast21		US East (Ohio) us-east-2		View analyzer for us-east-2
☐	stage-tf-bucketzhzast21		US East (Ohio) us-east-2		View analyzer for us-east-2

Tables (3) Info

Find tables

Any tag key

☐	Name	▲	Status	Partition key	Sort key	Indi
☐	terraform-lock		✔ Active	LockID (S)	-	
☐	terraform-lock-prod		✔ Active	LockID (S)	-	
☐	terraform-lock-stage		✔ Active	LockID (S)	-	